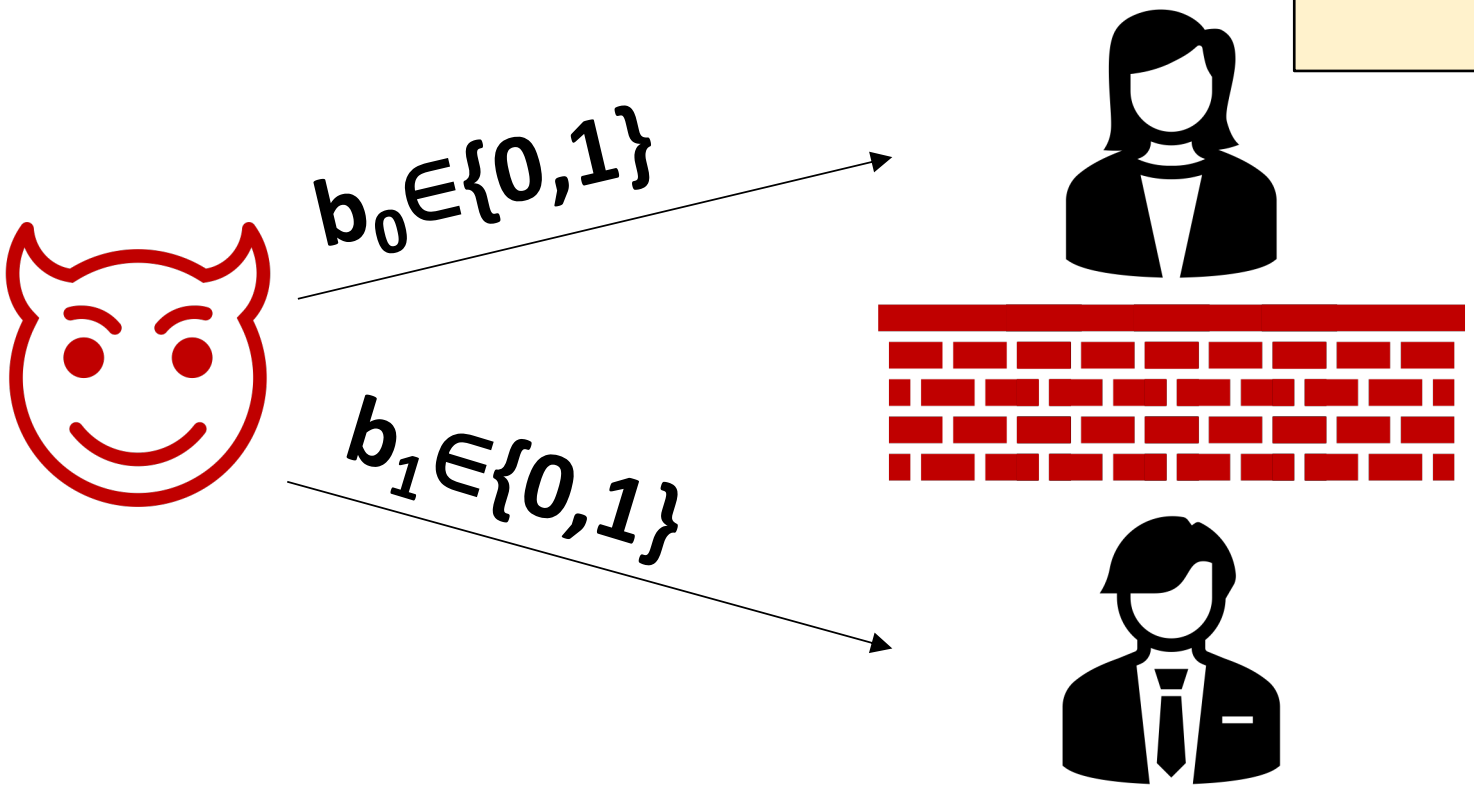
One-Shot Signatures

Mark Zhandry (Stanford University)

Based on joint works with Ryan Amos, Marios Georgiou, Aggelos Kiayias, and Omri Shmueli

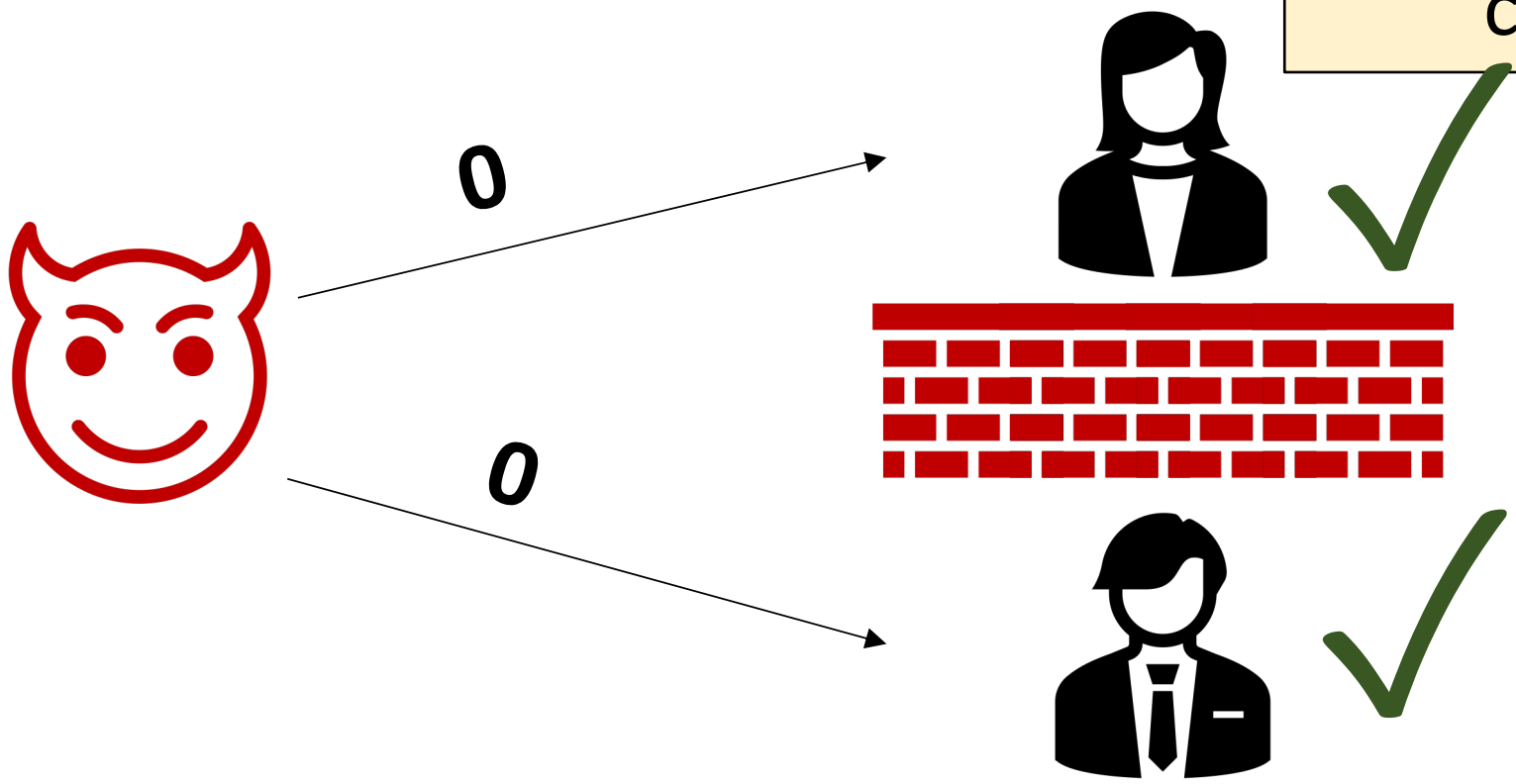
A question

Can Alice and Bob ensure that $\mathbf{b_0 = b_1}$ without any communication



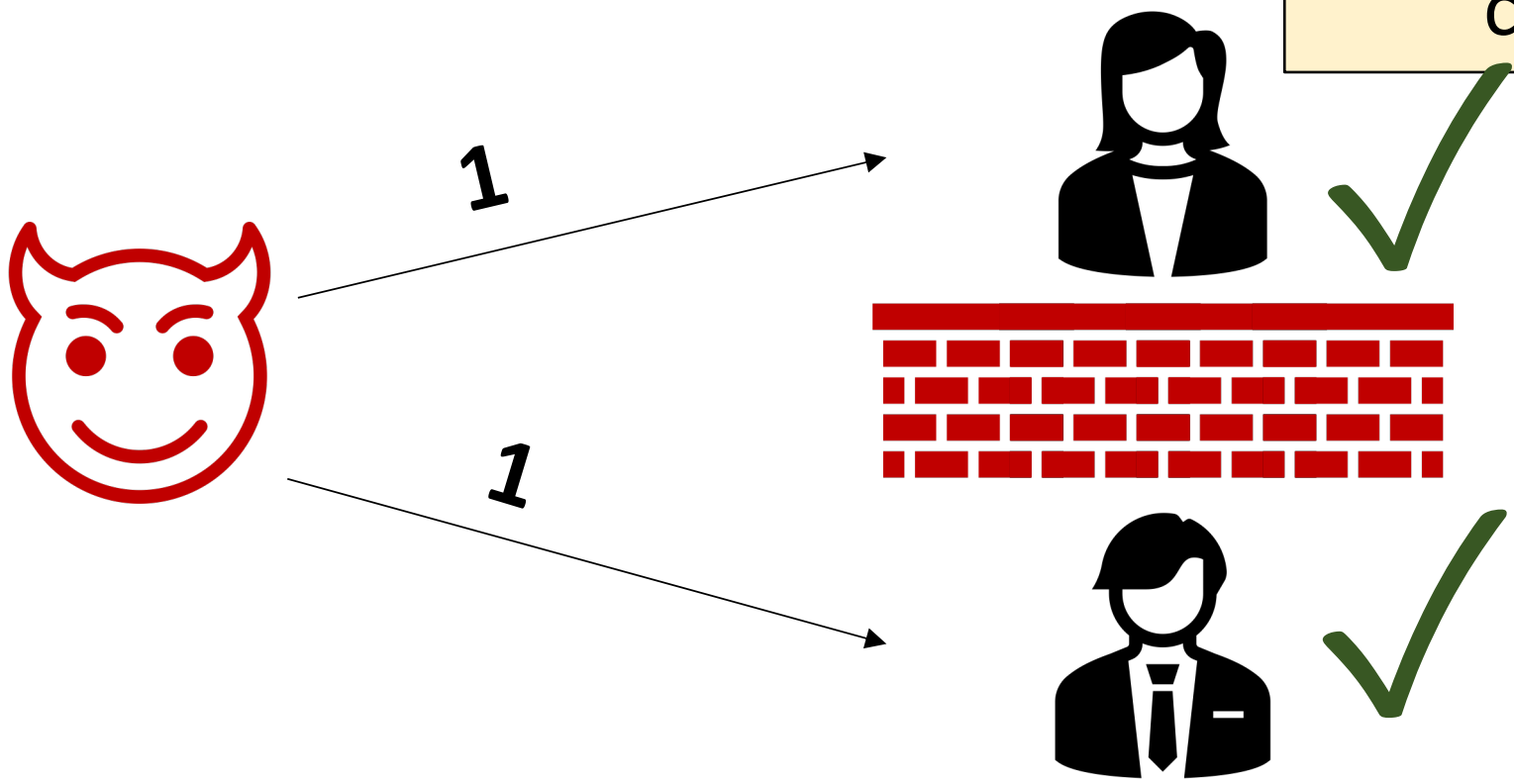
A question

Can Alice and Bob ensure that $\mathbf{b_0 = b_1}$ without any communication



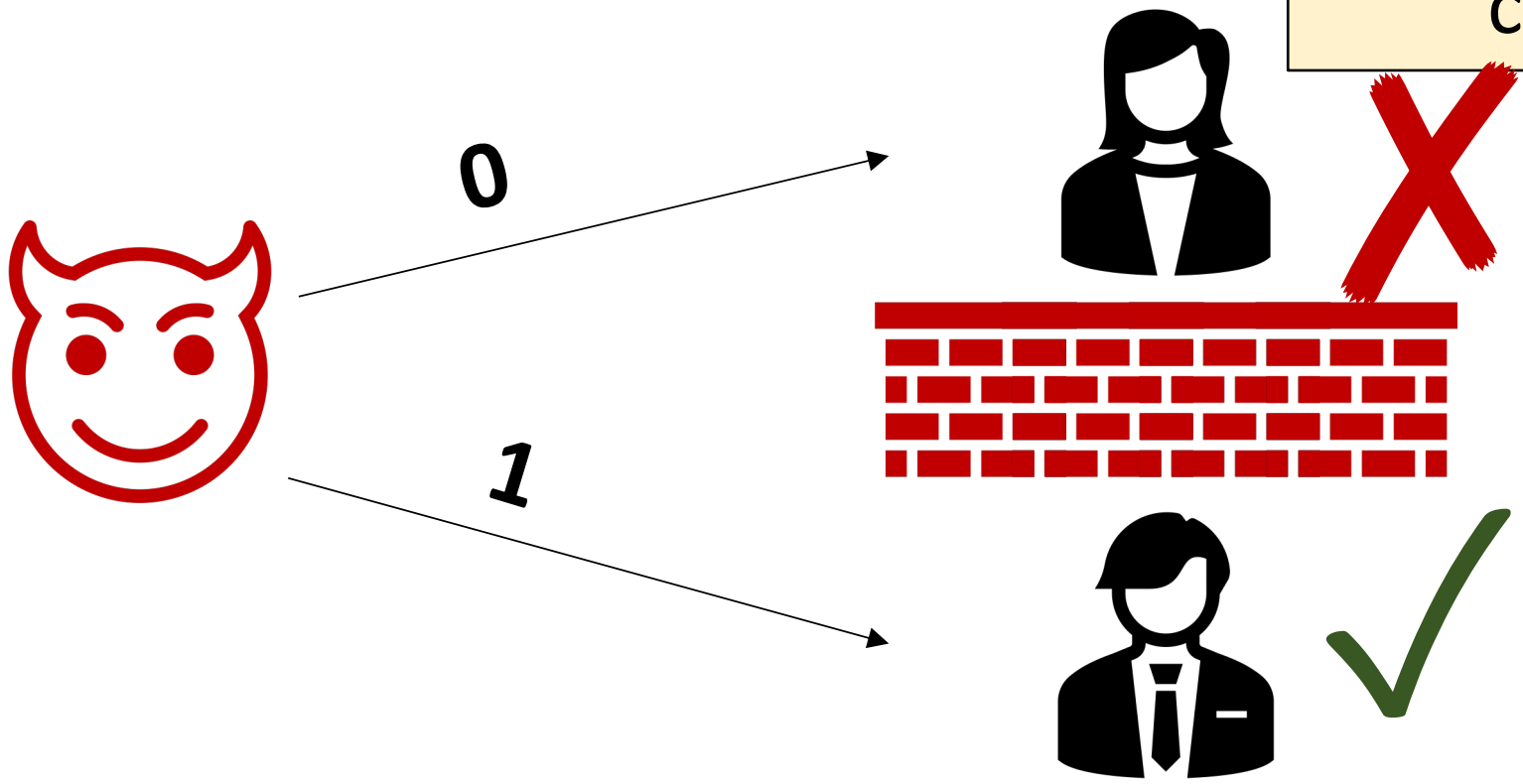
A question

Can Alice and Bob ensure that $\mathbf{b_0 = b_1}$ without any communication



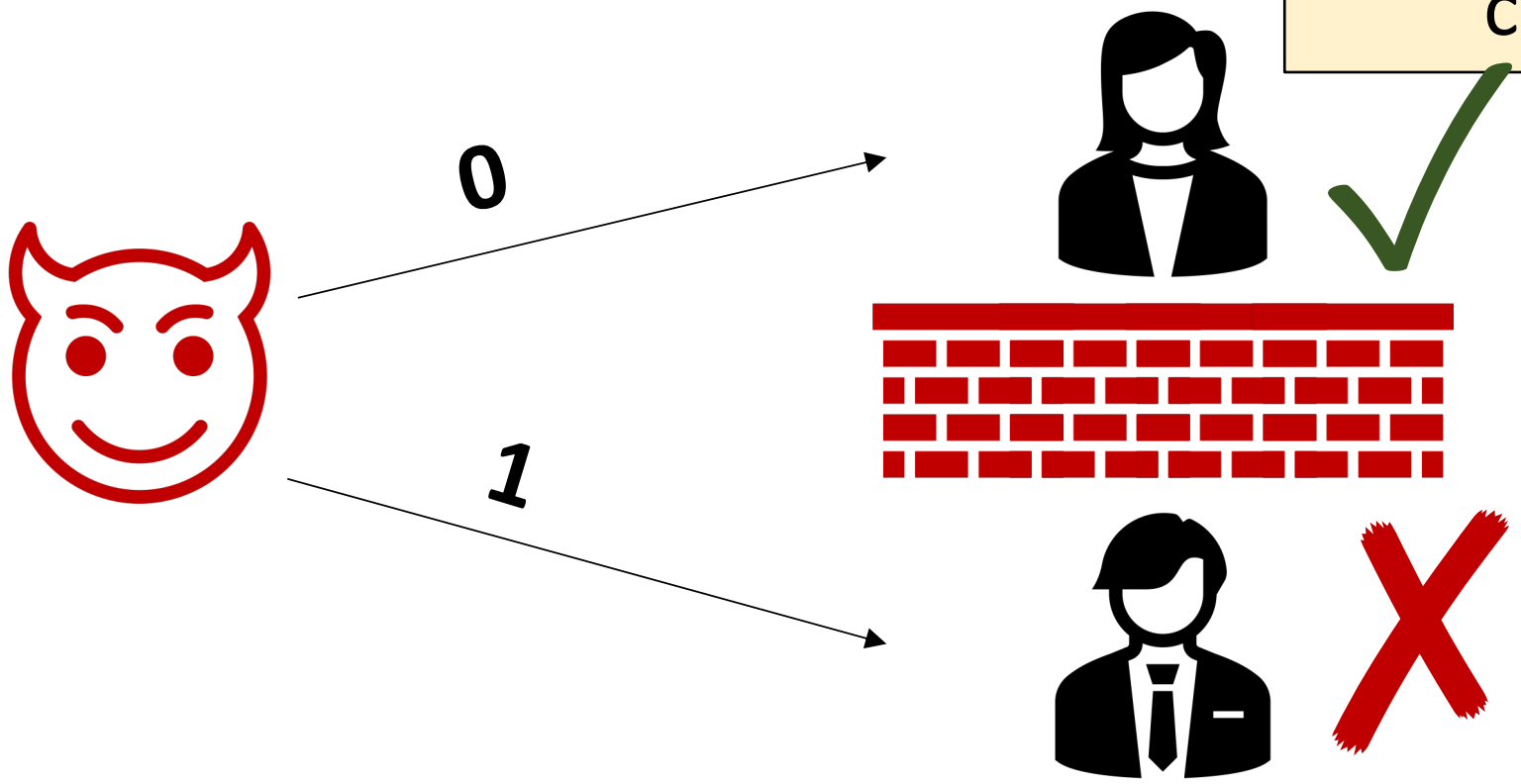
A question

Can Alice and Bob ensure that $\mathbf{b_0 = b_1}$ without any communication

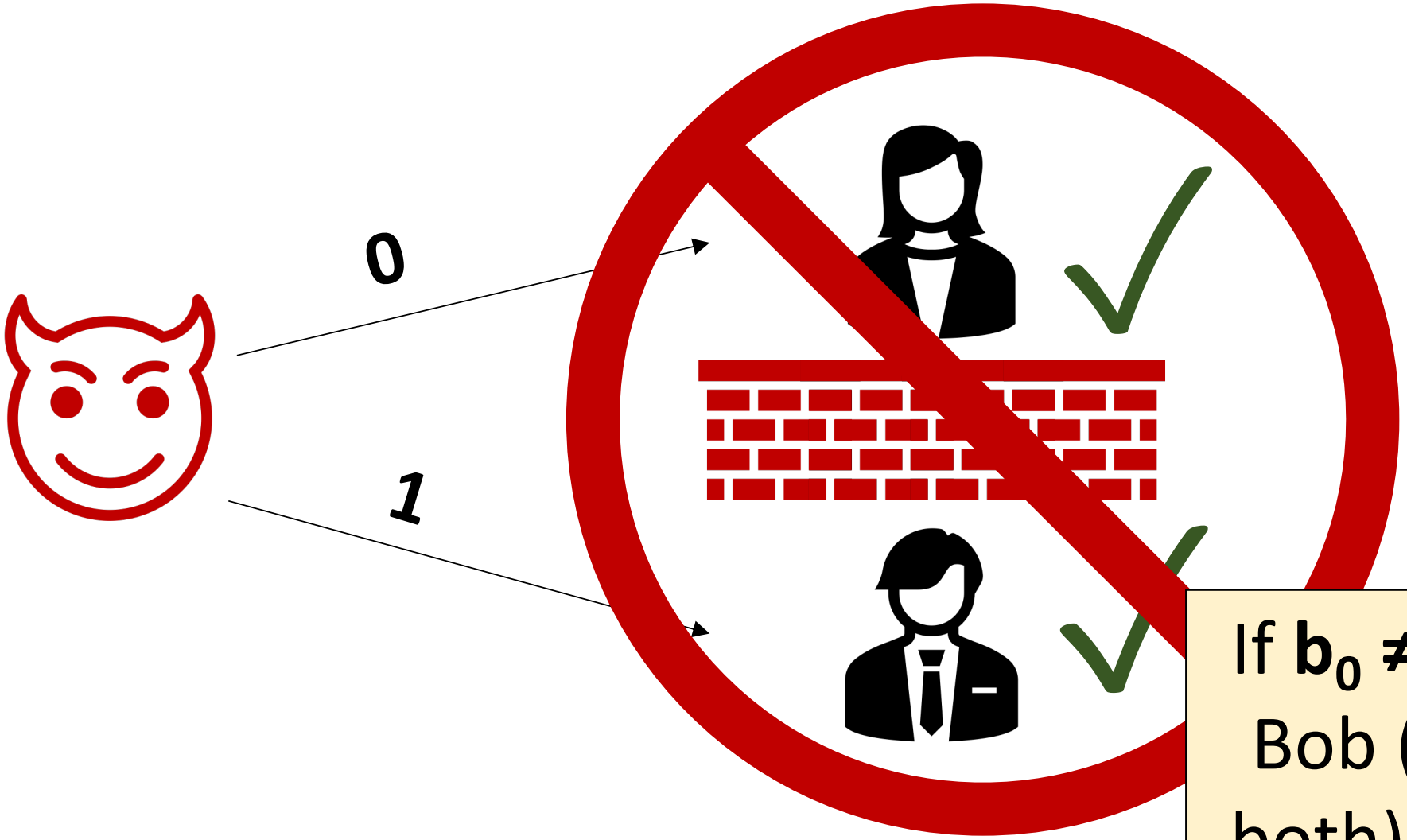


A question

Can Alice and Bob ensure that $\mathbf{b_0 = b_1}$ without any communication

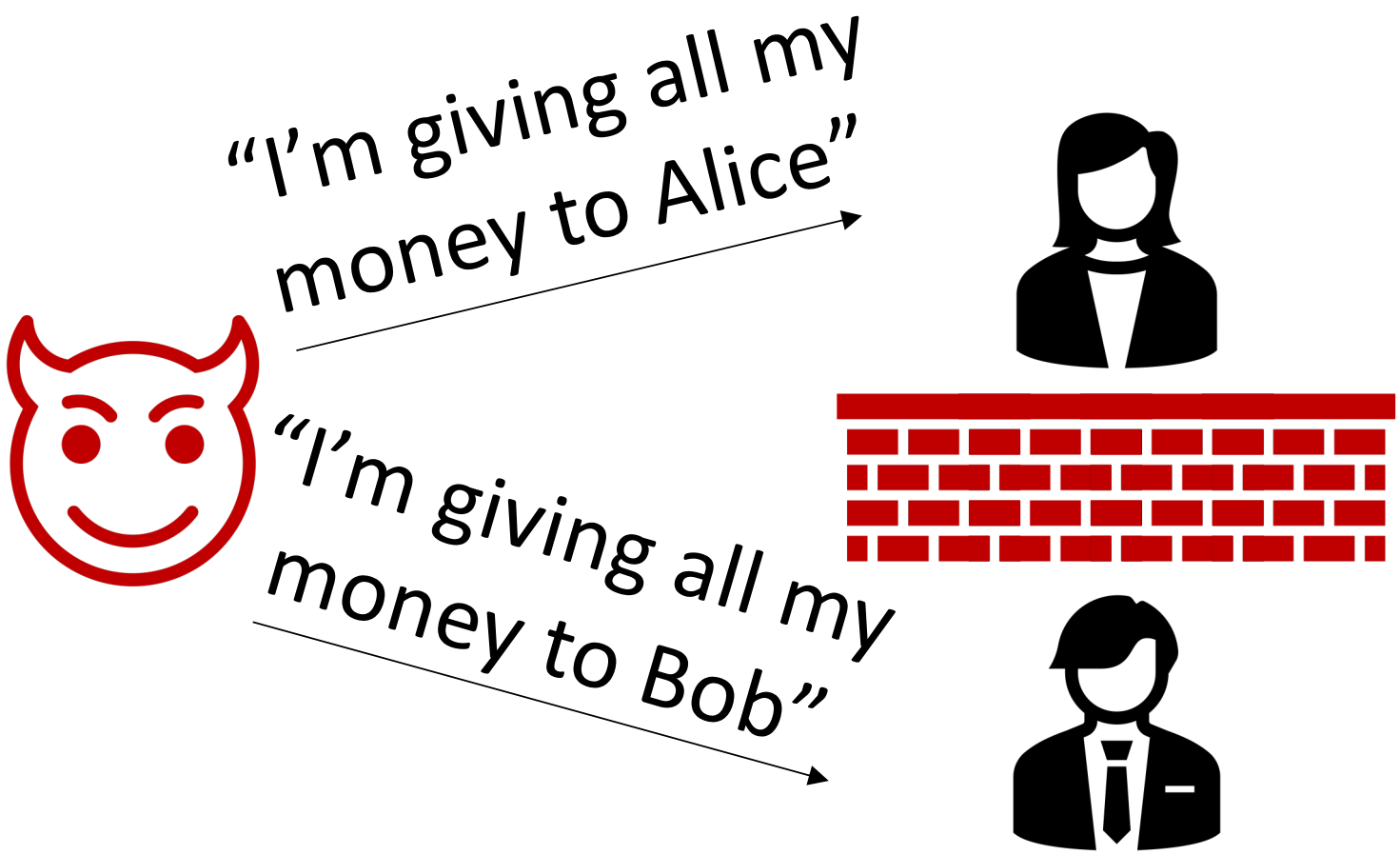


A question



If $\mathbf{b_0 \neq b_1}$, Alice or Bob (or possibly both) must reject

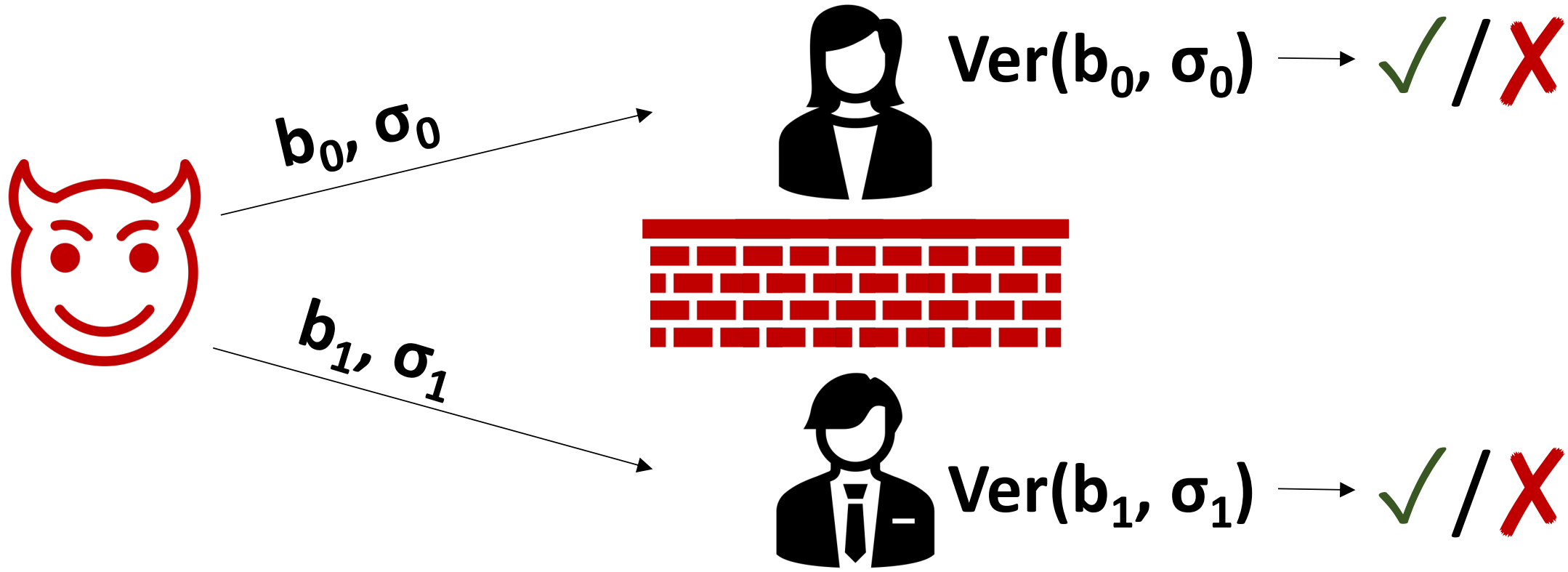
Ensuring consistency between parties is fundamental



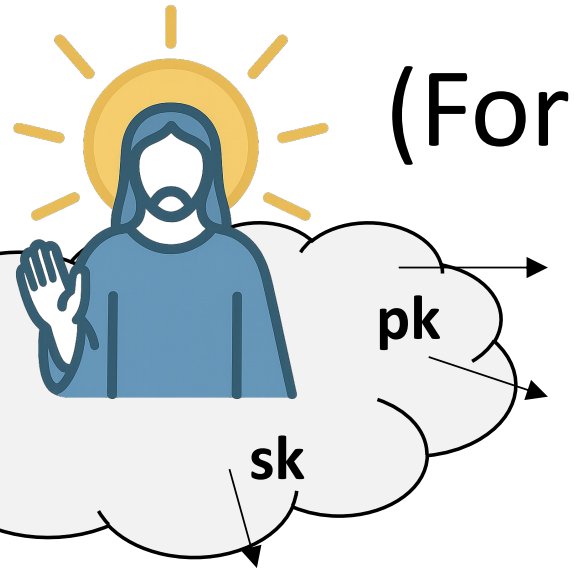
Of course, such consistency is trivially impossible, since Alice has no way to tell if $\mathbf{b_1 = b_0}$ or not

Right??????

A Slightly Less Preposterous Setup:



(For now)

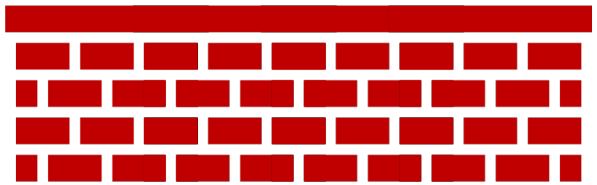


sk

b_0, σ_0

b_1, σ_1

pk



pk

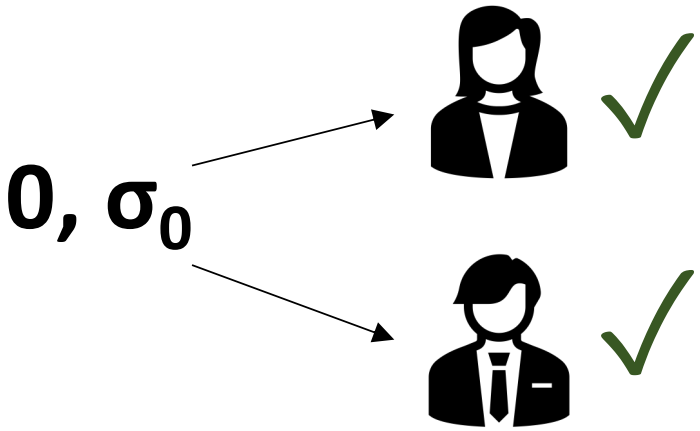
$\mathsf{Ver}(pk, b_0, \sigma_0) \rightarrow$ ✓ / ✗

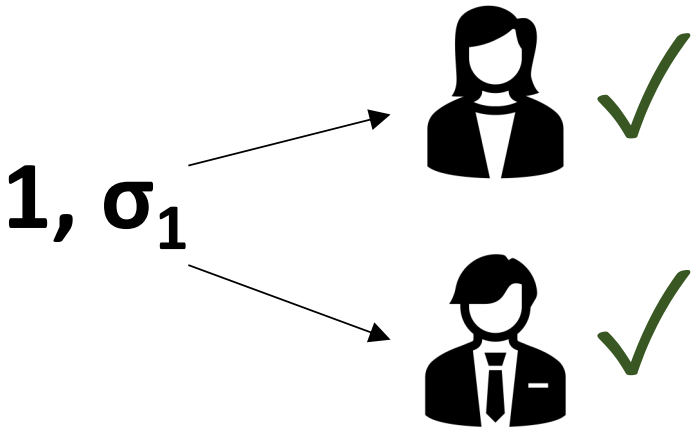
$\mathsf{Ver}(pk, b_1, \sigma_1) \rightarrow$ ✓ / ✗

$\sigma_i \leftarrow \mathsf{Sign}(sk, b_i)$

Such consistency is still impossible!

By the correctness of protocol, both of the following are possible:





 : simply create both **(0, σ_0)** and **(1, σ_1)**, send one to Alice, and one to Bob

If honest protocol is efficient, so is adversary (so crypto doesn't help)

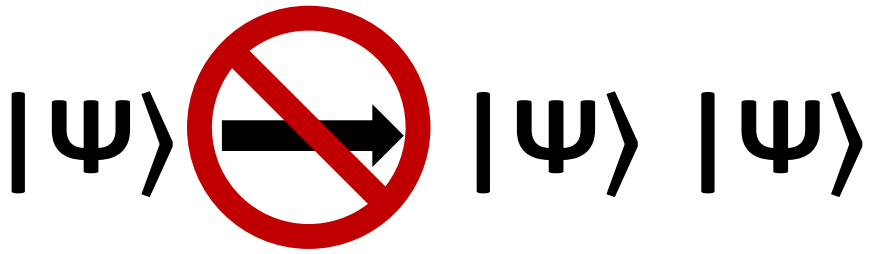
: simply create both $(0, \sigma_0)$ and $(1, \sigma_1)$, send one to Alice, and one to Bob

Requires being able to run the computation to produce σ_b twice. What if running it the first time consumes **sk**?

Doesn’t make sense classically, but…

Enter Quantum Mechanics

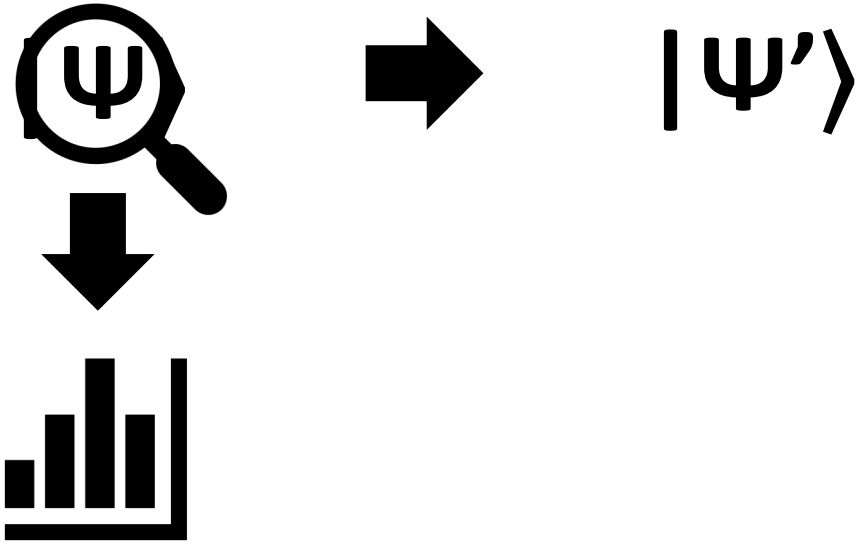
Quantum No Cloning



Contrast to classical information:

01001110 → 01001110
01001110

Observer Effect

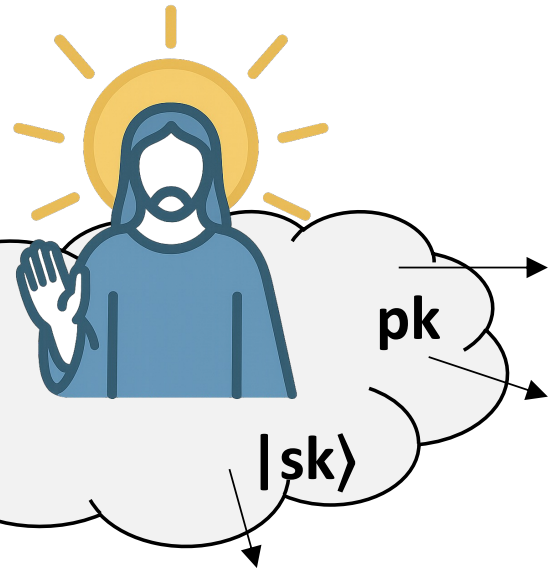


Contrast to classical information:

01001110 → 01001110

“there are four 1’s”

What if, by the observer effect, generating **σ** actually alters/consumes **|sk⟩**. Maybe it is impossible to ever sign a second message



$|sk\rangle$

b_0, σ_0

b_1, σ_1

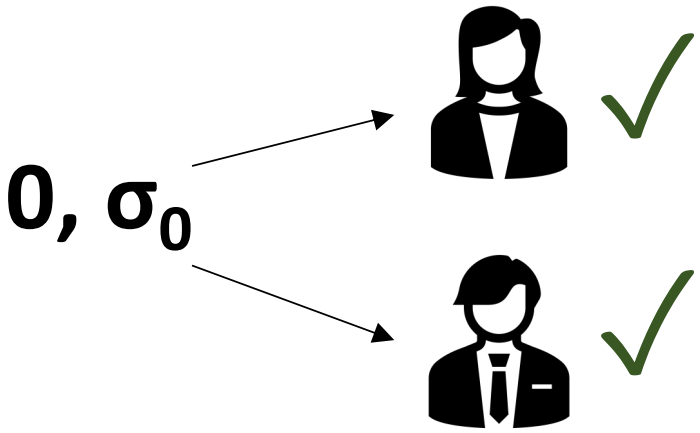
$\mathsf{Ver}(pk, b_0, \sigma_0) \rightarrow$ ✓ / ✗

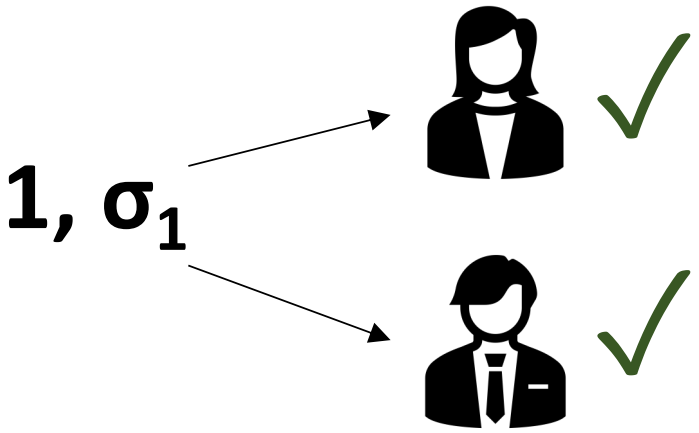
$\mathsf{Ver}(pk, b_1, \sigma_1) \rightarrow$ ✓ / ✗

$\sigma_i \leftarrow \mathsf{Sign}(|sk\rangle, b_i)$

Such consistency *still* impossible, statistically

By the correctness of protocol, both of the following exist:





: Simply brute-force search for $(0, \sigma_0)$ and $(1, \sigma_1)$, send one to Alice, and one to Bob

: Simply brute-force search for $\mathbf{(0, \sigma_0)}$ and $\mathbf{(1, \sigma_1)}$, send one to Alice, and one to Bob

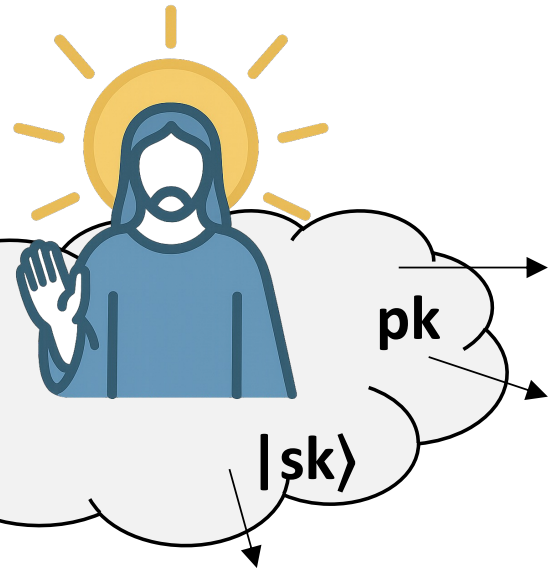
Requires time exponential in bit-length of $\boldsymbol{\sigma}$.

Maybe it's impossible for *efficient* computation…

Enter Quantum Cryptography

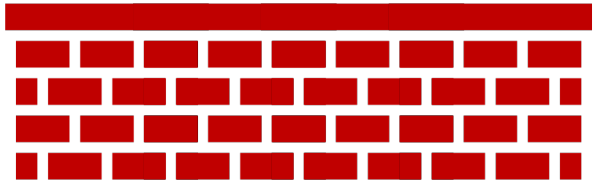
Quantum Signature Tokens

[BenDavid-Sattath'16]



b_0, σ_0

b_1, σ_1



$\mathrm{Ver}(\mathrm{pk}, b_0, \sigma_0) \rightarrow$ ✓ / ✗

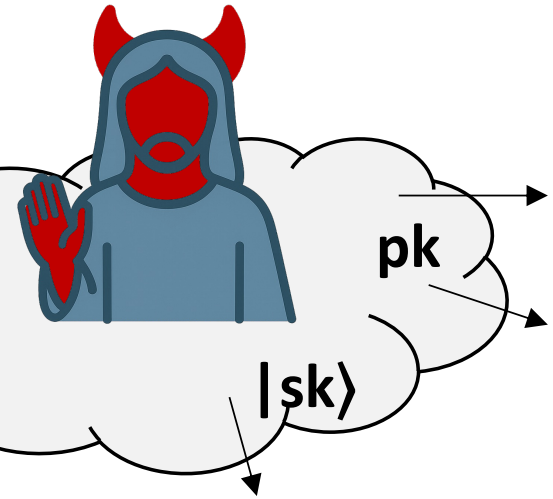
$\mathrm{Ver}(\mathrm{pk}, b_1, \sigma_1) \rightarrow$ ✓ / ✗

Computationally Efficient

What if we don’t want to trust the setup

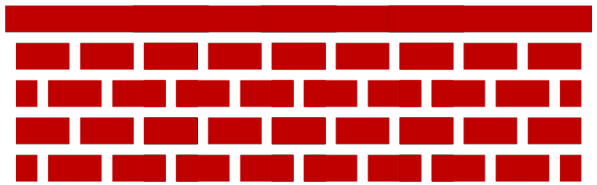
One-Sho

Only guarantee is that Alice and Bob receive the same **pk**



b_0, σ_0

b_1, σ_1



$\mathsf{Ver}(\mathsf{pk}, b_0, \sigma_0) \rightarrow$ ✓ / ✗

$\mathsf{Ver}(\mathsf{pk}, b_1, \sigma_1) \rightarrow$ ✓ / ✗

Computationally Efficient

OSS sounds absurd!

Attacker can't copy $|\mathbf{sk}\rangle$, even though he created it himself!

This does not obviously follow from traditional no-cloning theorem, which only applies to states that are unknown to would-be cloner

Numerous Applications

Digital currency secured by unclonability of quantum states

- No need for blockchain or other ledger to record transactions
- Can even send quantum banknotes over classical networks

Overcome a number of barriers in blockchain settings [Drake’23]

- E.g. multiple parties can reach consensus against a larger share of corrupted users

Numerous Applications

Demonstrates the weirdness of quantum computation

- Can’t keep a program trace
- Quantum algorithm can’t even know its internal state, even though it created its state itself

How do we actually build OSS?

[Unruh'16, implicit]: OSS relative to a *quantum* oracle

[Amos-Georgiou-Kiayias-Z'20]: OSS relative to a classical oracle, with heuristic instantiation

Except, our "proof" turned out to be flawed

[Shmueli-Z'25]: OSS in classical oracle model, or from somewhat standard cryptographic assumptions

[Huang-Shmueli-Vaikuntanathan-Z'25]: improved parameters

Main idea

Building block: a special type of collision-resistant hash function **H**

Key generation: (1) Create "uniform superposition" over domain of **H**
(2) Measure **H(x)** to get image **pk**
(3) State "collapses" to superposition over primages of **pk** → call this $|\mathbf{sk}\rangle$

Signature on a bit **b**: **x** s.t. **H(x) = pk** and $\mathbf{x}_1 = \mathbf{b}$

Main idea

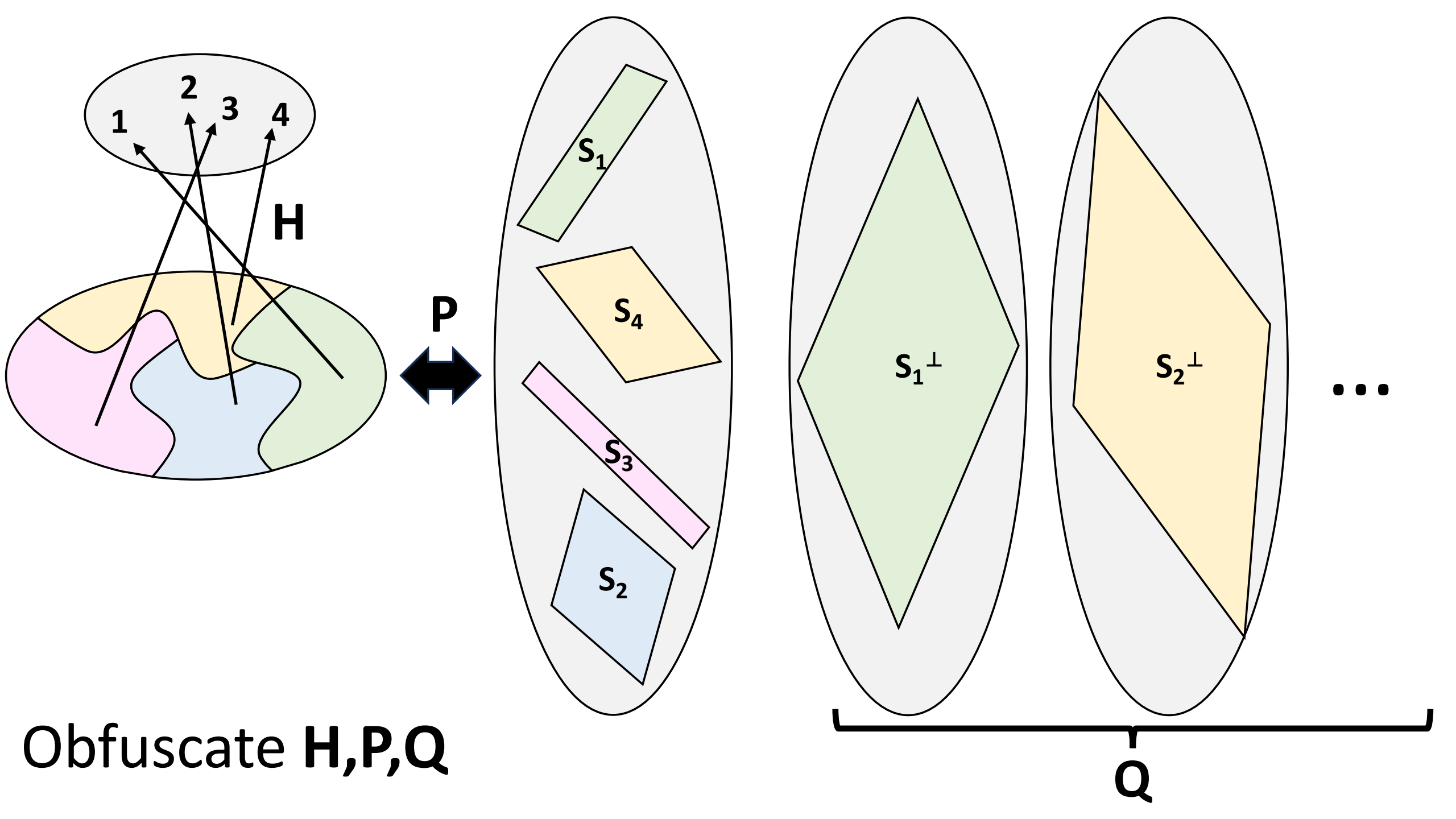
Signatures on both **0** and **1** give collision for **H**

➡ Security follows assuming **H** is collision-resistant

Secret key is a superposition of signatures on all bits

Signing must somehow turn this into
signatures over a given bit

Signing process must inherently destroy secret key



Obfuscate **H,P,Q**

P,Q allow for signing messages

This follows from somewhat standard techniques in unclonable cryptography, following [Aaronson-Christiano'12]

The hard part: proving the collision-resistance of **H**

Required many interesting ideas, and even lead to novel developments in the classical theory of obfuscating programs

OSS is still a highly theoretical object

(1) Need a fault-tolerant quantum computer

(2) We need very mathematical form of obfuscation

- highly impractical
- post-quantum security is not well-understood

Thanks!